

Nome: Nicolas Demantova Ribeiro

UNIARP – Caçador - 5º Semestre – Curso ADS

Professor: Edson

Matéria: Verificação e Validação de Software (VVS)



# RELATÓRIO TÉCNICO

## 1.1. Introdução

O mundo da tecnologia está em constante inovação. Diversas ferramentas e integrações diferentes são utilizadas para facilitar nossa vivência e transformar tarefas antes consideradas inconvenientes ou impossíveis acessíveis não apenas para profissionais na área, mas também a pessoas comuns. Tendo isso em foco a disciplina de Verificação e Validação de Software se torna indispensável para avaliar os limites de tecnologias ascendentes, bem como automatizar e simplificar rotinas que melhoram e corrigem problemas de forma simples e efetiva.

Para este trabalho foram utilizadas uma gama de ferramentas extremamente poderosas e fáceis de serem utilizadas, com o objetivo de trazer à tona a importância de um desenvolvimento responsável e efetivo. Abaixo segue um sumário das ferramentas escolhidas:

**Selenium:** Uma plataforma com drivers focados para automação de desenvolvimento web, focado nos browsers mais populares da atualidade, como Chrome e Firefox. O Selenium promete uma facilidade para automatizar testes de U.I, CRUD e até integração com diversas linguagens de programação modernas.

**PyTest:** Uma framework voltada para testes unitários utilizando a linguagem Python 3, o Pytest auxilia testar programas feitos em python de forma simples e rápida, podendo testes complexos serem escritos em pouquíssimas linhas. Com um pacote disponível no **PIP** o Pytest é facilmente baixado e usado em sua totalidade.

**Postman:** O Postman é uma aplicação centrada para testes de API. Com uma interface simples e completa, o Postman se torna uma ótima ferramenta para automatizar testes em grandes quantidades e garantir funcionamento de APIs escritas por um usuário ou até de grandes empresas.

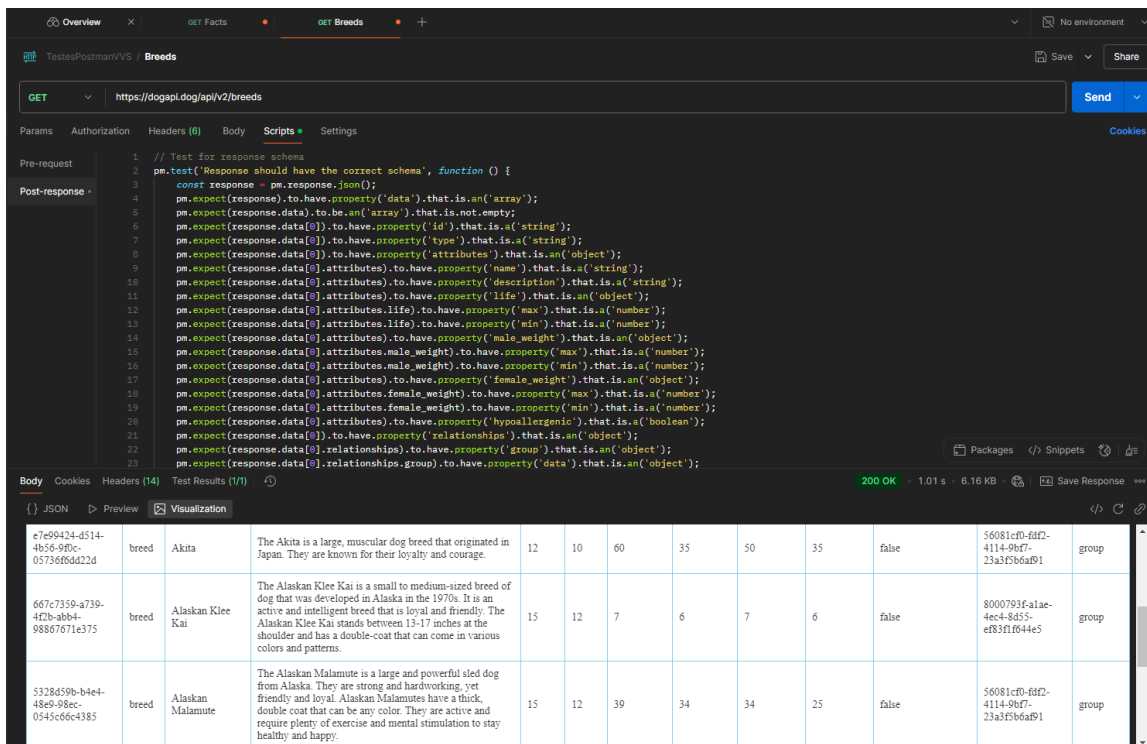
**Appium:** Aplicativos mobile se tornaram extremamente comuns em nosso dia-a-dia, e por isso vemos o surgimento de ferramentas como o Appium, que promete facilitar testes dentro de SDKs mobile como Android e Apple. O Appium conta com uma vasta quantidade de ferramentas que podem tranquilamente automatizar e repetir testes em aplicativos mobile.

## 1.2. Metodologia

**Selenium e Pytest:** Utilizando um VENV (Virtual Environment) e a integração do Selenium com o Pytest é realizado um teste simples de uma U.I feita em tabs no site <https://demoqa.com/tabs> . A seguir o código de teste completo:

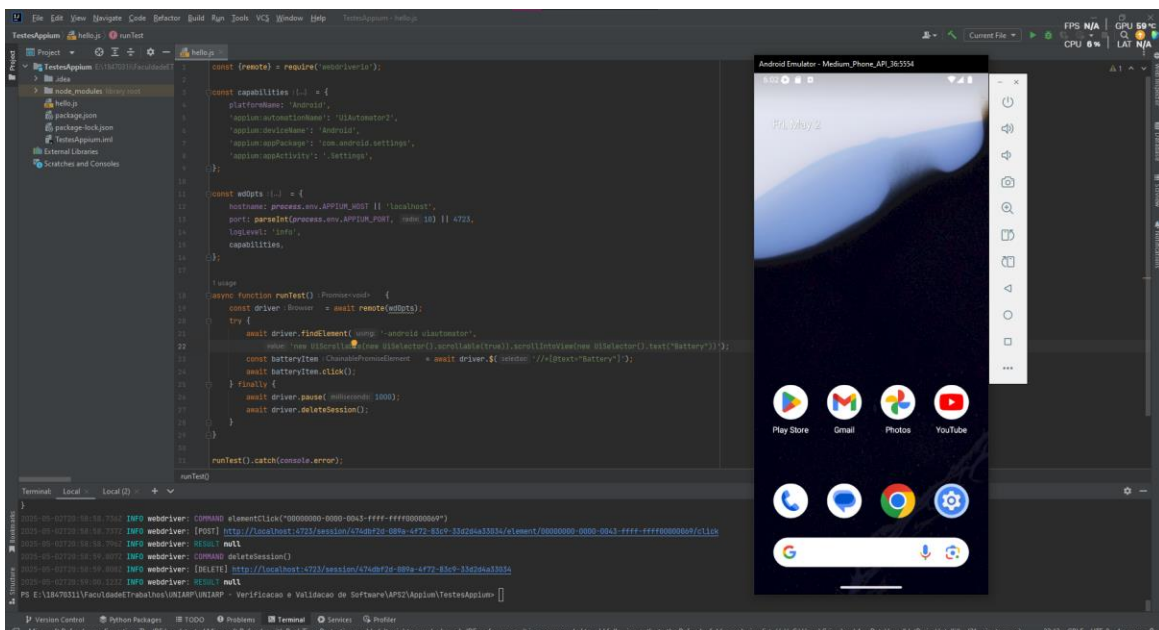
```
1  from selenium import webdriver
2      from selenium.webdriver import ActionChains
3      from selenium.webdriver.common.actions.wheel_input import ScrollOrigin
4      from selenium.webdriver.common.by import By
5  import pytest
6
7
8  def teste_tabs(): # Testa elementos de UI dispostos em tabs no DemoQA
9      driver = webdriver.Firefox()
10     driver.get("https://demoqa.com/tabs")
11     tab_what = driver.find_element(by=By.ID, value="demo-tab-what")
12     tab_origin = driver.find_element(by=By.ID, value="demo-tab-origin")
13     tab_use = driver.find_element(by=By.ID, value="demo-tab-use")
14     driver.implicitly_wait(0.5)
15     tab_what.click()
16     if not tab_what.get_attribute("aria-selected"):
17         driver.quit()
18         pytest.fail("Tab 'what' não selecionada! Teste falhado")
19     tab_origin.click()
20     if not tab_origin.get_attribute("aria-selected"):
21         driver.quit()
22         pytest.fail("Tab 'origin' não selecionada! Teste falhado")
23     tab_use.click()
24     if not tab_use.get_attribute("aria-selected"):
25         driver.quit()
26         pytest.fail("Tab 'use' não selecionada! Teste falhado")
27     driver.quit()
```

**Postman:** Como template foi utilizada uma API pública simples (no caso <https://dogapi.dog/api/v2/breeds>) que traz dados sobre raças de cachorros. Um script para formatação em tabela e validação das informações foi escrito e rodado com sucesso. Abaixo uma captura-de-tela demonstrando o teste feito com sucesso.



|                                      |       |                  |                                                                                                                                                                                                                                                                                                                |    |    |    |    |    |    |       |                                      |       |
|--------------------------------------|-------|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|----|----|----|----|-------|--------------------------------------|-------|
| c7c09424-d514-4b56-990c-0573666d22d2 | breed | Akita            | The Akita is a large, muscular dog breed that originated in Japan. They are known for their loyalty and courage.                                                                                                                                                                                               | 12 | 10 | 60 | 35 | 50 | 35 | false | 56081cf0-fd2-4114-9bf7-23a3f5b6a9f1  | group |
| 667c7359-4739-4f2b-ab84-98867671e375 | breed | Alaskan Klee Kai | The Alaskan Klee Kai is a small to medium-sized breed of dog that was developed in Alaska in the 1970s. It is an active and intelligent breed that is loyal and friendly. The Alaskan Klee Kai stands between 13-17 inches at the shoulder and has a double-coat that can come in various colors and patterns. | 15 | 12 | 7  | 6  | 7  | 6  | false | 8000793f-a1ae-4ec4-8d55-ef83f1f044e5 | group |
| 5328d59b-b4e4-48e0-99ec-0545c66c4385 | breed | Alaskan Malamute | The Alaskan Malamute is a large and powerful sled dog from Alaska. They are strong and hardworking, yet friendly and loyal. Alaskan Malamutes have a thick, double coat that can be any color. They are active and require plenty of exercise and mental stimulation to stay healthy and happy.                | 15 | 12 | 39 | 34 | 34 | 25 | false | 56081cf0-fd2-4114-9bf7-23a3f5b6a9f1  | group |

**Appium:** Apesar de certas dificuldades em preparar o ambiente de testes, incluindo tanto as dependências do kit de desenvolvimento (SDK) Java e Android, o Appium proporcionou uma interface concisa e simples para realizar tarefas com apenas poucas linhas. O código em si realiza um teste de acessar o sumário da bateria dentro do app de configurações explícito no Android 16.0. Abaixo uma captura-de-tela mostrando tanto o código de teste completo quanto o dispositivo rodando através do Android Studio



```

1  const (results) = require('webdriverio');
2
3  const capabilities = {
4    platformName: 'Android',
5    'webdriver:browserName': 'WebDriver2',
6    'appium:deviceName': 'Android',
7    'appium:appPackage': 'com.android.settings',
8    'appium:appActivity': '.Settings',
9  };
10
11  const wdopts = {
12    hostname: process.env.APPIUM_HOST || 'localhost',
13    port: parseInt(process.env.APPIUM_PORT, 10) || 4730,
14    logLevel: 'info',
15    capabilities,
16  };
17
18  usage
19
20  async function runTest() {
21    const driver = await results.remote(wdopts);
22    try {
23      await driver.findElement(using: 'android:uiAutomator',
24        {
25          'new UiSelector().new UiSelector().new UiSelector().new UiSelector().text("Battery")');
26      const batteryItem = (await driver.$(webdriver: '//id=com.android.settings:id/battery')).element();
27      await batteryItem.click();
28    } finally {
29      await driver.pause(milliseconds: 1000);
30      await driver.deleteSession();
31    }
32  }
33
34  runTest().catch(console.error);
35
36  runTest();

```

Terminal output:

```

INFO webdriver: COMMAND element('new UiSelector().new UiSelector().new UiSelector().new UiSelector().text("Battery")')
INFO webdriver: [POST] http://localhost:4730/session/4730/element/0/00000000-0000-0043-ffff-ffff00000000/click
INFO webdriver: RESULT null
INFO webdriver: COMMAND deleteSession()
INFO webdriver: [DELETE] http://localhost:4730/session/4730/4730-0000-4730-0000-0000-0000-0000-0000

```

### 1.3. Tabela Comparativa

| <b>Critério</b>        | <b>Selenium</b>                                       | <b>Postman</b>                                | <b>Appium</b>                            | <b>Pytest</b>                                        |
|------------------------|-------------------------------------------------------|-----------------------------------------------|------------------------------------------|------------------------------------------------------|
| Tipos de testes        | Testes de interface gráfica (UI), E2E                 | Testes de API (REST, SOAP, GraphQL)           | Testes móveis (UI, E2E)                  | Testes unitários, integração, API, E2E               |
| Linguagens Suportadas  | Java, Python, C#, Ruby, JS, Kotlin, etc.              | Scripts em JavaScript (Pre/Post request)      | Java, Python, Ruby, JS, etc.             | Python                                               |
| Open-source?           | Sim                                                   | Sim (Postman Runtime via Newman)              | Sim                                      | Sim                                                  |
| Facilidade de Uso      | Média (exige programação)                             | Alta (interface gráfica amigável)             | Média (exige configuração e codificação) | Alta (fácil para quem já usa Python)                 |
| Integração CI/CD       | Boa (Jenkins, GitHub Actions, etc.)                   | Boa (via Newman CLI)                          | Boa                                      | Excelente (plugins e integração nativa com CI/CD)    |
| Geração de Relatórios  | Com ferramentas externas (Allure, TestNG, etc.)       | Sim (Relatórios visuais e via Newman com CLI) | Com ferramentas externas (Allure, etc.)  | Sim (built-in e via plugins como Allure, HTML, etc.) |
| Velocidade de Execução | Média                                                 | Alta                                          | Lenta (especialment e em emuladores)     | Alta                                                 |
| Suporte a Paralelismo  | Sim (com ferramentas como TestNG, pytest-xdist, etc.) | Limitado (via Newman + scripts paralelos)     | Sim (com frameworks de terceiros)        | Sim (pytest-xdist, pytest-parallel)                  |
| Cross-Browser/Cross-OS | Sim                                                   | Não aplicável                                 | Sim (Android, iOS, Windows, macOS)       | Sim (dependendo do teste e ambiente)                 |
| Licença                | Apache 2.0                                            | Proprietária (GUI) / Apache 2.0 (Newman)      | Apache 2.0                               | MIT                                                  |

## 1.4. Conclusão

A abrangência de ferramentas voltadas para o uso de Verificação e Validação de Software é considerável, e portanto é difícil escolher uma única considerando que muitas carregam tanto vantagens e desvantagens. Através desse trabalho foi possível ter uma compreensão maior sobre a necessidade de proficiência em programação para testes, bem como uma ideia firme e sólida dos requisitos necessários para testar softwares. Dito isso, a ferramenta que mais se destacou das escolhidas foi sem dúvida o **Selenium**. Seus casos de uso não apenas eram vários, mas também bem documentados, sem falar na facilidade de escrever scripts simples porém poderosos que tornariam os casos de testes necessários concisos e bem-estruturados. Também foi possível realizar uma integração com o **PyTest** para utilizar o funcionamento de ambas ferramentas simultaneamente. O resultado foi um teste aparentemente simples mas também fácil de ser expandido caso necessário.

Outra ferramenta que se destacou também foi o **Postman**. Apesar de ser relativamente mais simples e ter casos de uso limitados foi extremamente fácil de configurar e realizar os testes necessários. A interface do app para Windows é muito bem feita e elegante. Através do uso da A.I original deles, o Postbot, também é possível gerar scripts quase instantaneamente para casos de uso simples e diretos.

Por último falemos do **Appium**. A ferramenta promete uma maneira poderosa de integrar com testes mobile, principalmente nos Sistemas Operacionais (OS) mais populares (Android e Apple). Porém, como várias coisas relacionadas a celulares, não foi nada fácil implementar e preparar o ambiente de desenvolvimento. Muitos pacotes não estavam disponíveis diretamente e foi necessário uma grande quantidade de trabalho braçal para ter certeza que tudo estaria funcionando direito. Apesar de tudo foi possível escrever um teste simples dentro de uma IDE e rodá-lo sem muitos problemas.

Uma ótima semana,

- Nicolas